



# KỸ THUẬT LẬP TRÌNH

## Chương 2: Ngôn ngữ lập trình C

# Nội dung

---

1. Các khái niệm cơ bản
2. Các kiểu dữ liệu
3. Các lệnh điều khiển
4. Bài tập thực hành

# Giới thiệu

---

- Năm 1970 Ken Thompson sáng tạo ra ngôn ngữ B (Basic Combined Programming Language) trên môi trường HĐH Unix của máy DEC PD-7
- Năm 1972 Dennis Ritchie ở Bell Laboratories cùng Ken Thompson sáng tạo ra ngôn ngữ C nhằm tăng hiệu quả cho ngôn ngữ B.
- Ngôn ngữ C nhanh chóng phổ biến rộng rãi và được sử dụng để viết nên HĐH đa nhiệm UNIX, O/S 2, và ngôn ngữ Dbase.
- Phát triển thành C++ vào năm 1983 hỗ trợ lập trình hướng đối tượng
- Có nhiều trình biên dịch C khác nhau
  - Turbo C (từ ver 1 tới ver 3), Microsoft C (ver 1 tới ver 6)
  - C++ với 3 trình biên dịch nổi tiếng Borland C++, Visual C++, và Turbo C++
- **Chúng ta sẽ sử dụng:**
  - Borland C++ 5.02 hoặc C-Free hoặc Dev C++

# Tập ký tự

---

❖ Mọi ngôn ngữ đều được xây dựng trên 1 bộ ký tự. Bộ ký tự trong C gồm:

- Các chữ cái in hoa: A, B, C, ..., Z
- Các chữ cái thường: a, b, c, ..., z
- Các chữ số: 0, 1, 2, ..., 9
- Các loại dấu chấm câu . ! ? : ;
- Các dấu toán học: + , - , \* , / , % , > , < , = , ( , )
- Các dấu đặc biệt: @ , # , & , A , \$ { , } , [ , ] , \ , |
- Các ký tự không nhìn thấy: Space, Enter, Tab
- Dấu gạch dưới \_

# Từ khóa

---

- Là các từ dùng riêng cho ngôn ngữ
- Từ khoá phải được dùng đúng cú pháp quy định
- Mọi từ khoá đều viết thường (không viết in hoa)
- Các từ khoá thường gặp.

auto  
do  
goto  
short  
union

break  
double  
if  
sizeof  
unsigned

case  
else  
int  
static  
void

char  
extern  
long  
struct  
volatile

continue  
float  
register  
switch  
while

default  
for  
return  
typedef

# Tên và cách đặt tên

---

- Dùng để định danh các thành phần của chương trình
- Tên biến, tên hàm, tên hằng, file, cấu trúc...
- Gồm chữ số dấu gạch nối “\_”
- Độ dài tối đa 32
- Lưu ý:
  - không được chứa kí tự trống (space)
  - không được bắt đầu bằng một chữ số
  - không được trùng với từ khóa
  - Không đặt tên ở dạng số mũ hoặc chỉ số (vd:  $H_2SO_4$ )
  - Không đặt tên với ký tự tiếng việt (vd: SốMaX, TổngChẵn...v.v)
- Nên đặt tên một cách gợi nhớ có ý nghĩa.
- Tên chuẩn: một số tên có sẵn của trình biên dịch.
- C là ngôn ngữ phân biệt viết hoa, viết thường

# Tên và cách đặt tên

---

- Cách đặt tên nào đúng:

- a) Hoc-tap
- b) Bien2
- c) \_HangSo
- d) 2SinhVien
- e) Lớp
- f) char
- g) Sinh\_vien
- h) Mon hoc
- i) Ngay,sinh

# Hằng, Biến, Biểu thức

---

## ❖ Hằng:

- Là đại lượng có giá trị không thay đổi được
- Ví dụ:
  - 124 Là một **hằng số**
  - 'D' Là một **hằng ký tự**
  - "Lap trinh" Là một **hằng chuỗi ký tự**

## ❖ Biến

- Là đại lượng có thể thay đổi được giá trị (gán giá trị mới)

## ❖ Biểu thức

- Là công thức tính toán để có 1 giá trị theo quy tắc toán học
- Gồm các toán hạng và các phép toán (toán tử)
  - Toán hạng: **hằng, biến, hàm hoặc biểu thức khác.**
  - Phép toán: **Số học, luận lý, gán, điều kiện, lấy địa chỉ, tăng giảm**



# Các phép toán C hỗ trợ

---

- Số học: `+` , `-` , `*` , `/` , `%`
- Luận lý: `==` , `!=` , `>` , `<` , `>=` , `<=` , `&&` , `||` , `!`
- Gán giá trị: `=`
- Lấy kích cỡ theo số byte: `sizeof(đồitượng)`
- Điều kiện: `ĐK ? E1 : E2`
- Lấy địa chỉ: `&(biến)`
- Tăng giảm: `x++` , `++x` , `x--` , `--x`  
`tong+=i;`      `tich*=i;`  
`%=` , `/=` , `-=` , `&=` , `|=` , `^=`

# Các hàm toán học thông dụng

Chỉ thị tiền biên dịch:

```
#include <math.h>
```

|                        |                                  |
|------------------------|----------------------------------|
| <code>sqrt(x)</code>   | lấy căn bậc 2                    |
| <code>exp(x)</code>    | lấy $e^x$                        |
| <code>log(x)</code>    | lấy $\log_e x$                   |
| <code>abs(x)</code>    | lấy $ x $ (kết quả là số nguyên) |
| <code>fabs(x)</code>   | lấy $ x $ (kết quả là số thực)   |
| <code>pow(x, y)</code> | lấy $x^y$                        |
| <code>ceil(x)</code>   | lấy số nguyên gần x nhất         |

- Các biểu thức phức tạp phải **sử dụng biến đổi toán học** để có được biểu diễn tương ứng trong C.

# Khai báo hằng

---

## Cú pháp

`#define TÊN_HẰNG giá trị`  
hoặc  
`const kiểu_dữ_liệu TÊN_HẰNG = giá trị;`

## Ví dụ

```
#define MAX 100  
const int SOPT = 100;
```

**Lưu ý:** hằng phải được khai báo trước khi sử dụng

# Khai báo biến

---

- Cú pháp

```
Kiểu dữ liệu TÊNBIẾN;
```

```
Kiểu dữ liệu Biến1, Biến2;
```

```
Kiểu dữ liệu TÊNBIẾN = giá trị ban đầu;
```

- Ví dụ

```
int sole;
```

```
float tbc, tong;
```

```
char Enter = '\n';
```

- **Lưu ý:** Biến có thể khai báo bất kỳ đâu trong chương trình miễn là trước khi sử dụng.

# Phép gán giá trị

- Cú pháp

**TênBiến = Biểu thức giá trị ;**

- Ví dụ:

```
int x, y, z;  
x=0;  
50=x; //sai  
3+y=x; //sai  
Z=x+y;  
x+y=z; //sai
```

# Câu lệnh, Chú thích

**Câu lệnh:** Là một chỉ thị yêu cầu máy tính thực thi một tác vụ nào đó. Một câu lệnh phải kết thúc bởi một dấu chấm phẩy “;”

## **Chú thích:**

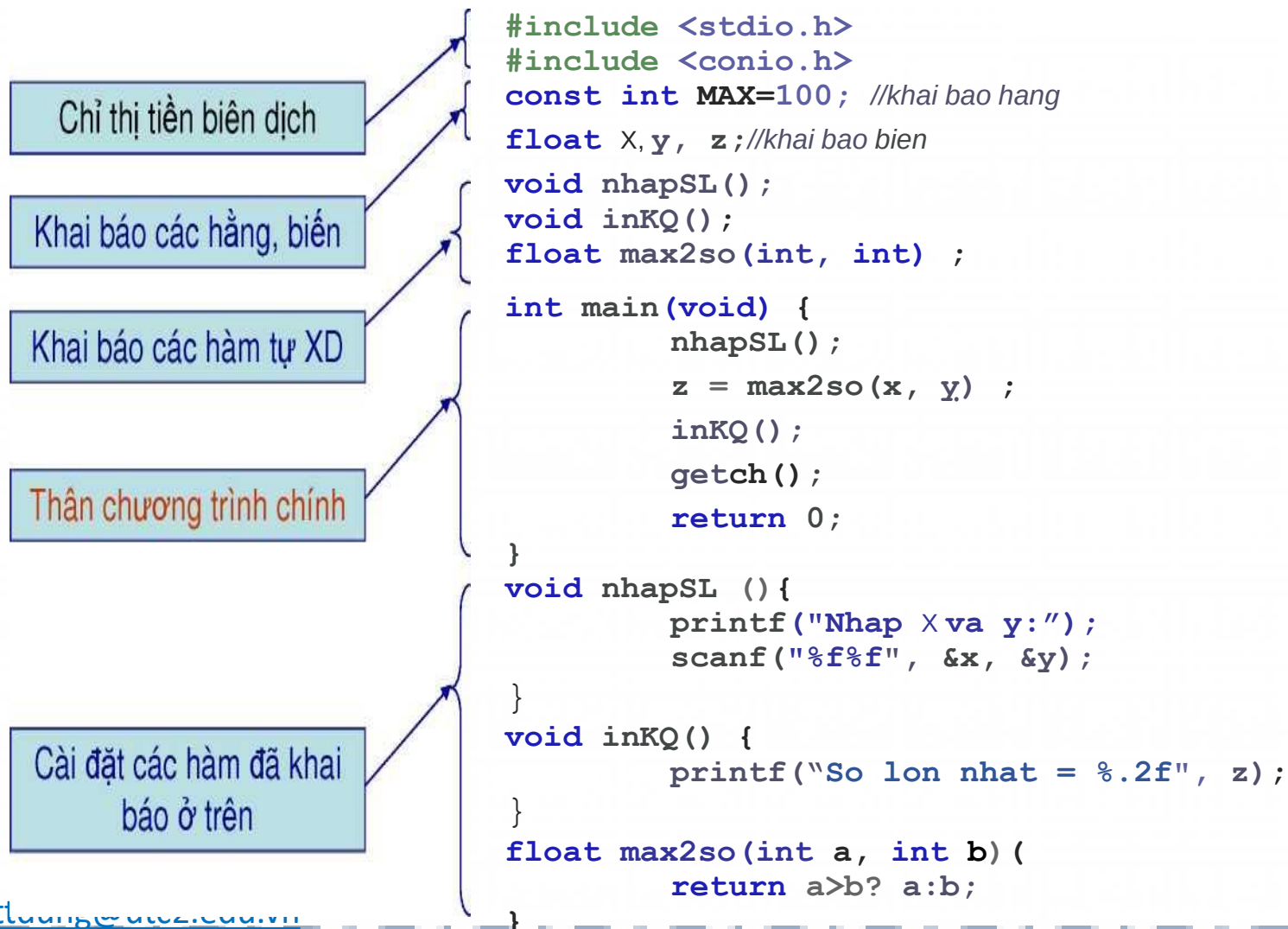
- Trên 1 dòng: // Chú thích trên một dòng
- Trên nhiều dòng:

/\*  
Chú thích trên nhiều dòng.  
\*/

## **Các chỉ thị tiền biên dịch:**

|         |           |          |        |
|---------|-----------|----------|--------|
| #define | #undefine | #include |        |
| #if     | #elif     | #else    | #endif |
| #ifdef  | #else     | #endif   |        |
| #ifndef | #else     | #endif   |        |
| #error  |           |          |        |

# Cấu trúc 1 chương trình C



# Một số quy tắc cần nhớ

---

- Đặt tên biến, hằng, hàm,... một cách **gợi nhớ**
- Khi sử dụng biến, hằng cần kiểm **tra lại** xem biến hằng đó đã được **khai báo** trước đó hay chưa ?
- C là ngôn ngữ phân biệt hoa/thường => phải **kiểm tra kỹ từng ký tự** khi viết
- Loại bỏ các chỉ thị **#include <file \*.h>** không cần thiết (để lại sẽ làm chậm quá trình biên dịch và tăng kích thước file \*.exe sau biên dịch)
- Viết chương trình theo cấu trúc khối (**phân cấp thụt đầu dòng**)
- Chương trình viết xong chưa thể biết được đúng hay sai => **phải chạy thử** với các bộ số liệu khác nhau (nên test thử với các **bộ số liệu đặc biệt**)



# Các kiểu dữ liệu trong C

---

- Các kiểu dữ liệu cơ bản
  - Số
  - Ký tự
  - Chuỗi ký tự
  - Mảng
- Ép kiểu trong C
- Nhập xuất dữ liệu trong C

# Các kiểu dữ liệu cơ bản

- Kiểu số:

| Tên kiểu      | Độ lớn  | Phạm vi                                |
|---------------|---------|----------------------------------------|
| char          | 1 byte  | -128 $\rightarrow$ 127                 |
| unsigned char | 1 byte  | 0 $\rightarrow$ 255                    |
| int           | 2 byte  | -32768 $\rightarrow$ 32767             |
| unsigned int  | 2 byte  | 0 $\rightarrow$ 65535                  |
| short         | 2 byte  | 0 $\rightarrow$ 65535                  |
| long          | 4 byte  | - $2^{31} \rightarrow 2^{31} - 1$      |
| unsigned long | 4 byte  | - $2^{31} \rightarrow 2^{32} - 1$      |
| float         | 4 byte  | $3.4e^{-38} \rightarrow 3.4e^{38}$     |
| double        | 8 byte  | $1.7e^{-308} \rightarrow 1.7e^{308}$   |
| long double   | 10 byte | $3.4e^{-4932} \rightarrow 1.1e^{4932}$ |

# Các kiểu dữ liệu cơ bản

- Kiểu ký tự:

| Tên kiểu | Độ lớn | Phạm vi                                              |
|----------|--------|------------------------------------------------------|
| char     | 1 byte | Gồm 256 ký tự có mã từ 0 → 255 ( xem Bảng mã ASCII ) |

- Ví dụ:

- Ký tự 'A' có mã 65
- Ký tự '0' có mã 48

- Các ký tự đặc biệt:

- '\0' : ký tự Null (có mã = 0) ký tự
- '\n' : xuống dòng (new line) ký tự
- '\t' : Tab

# Bảng mã ASCII

| ASCII control characters |      |                       | ASCII printable characters |       |     | Extended ASCII characters |     |      |
|--------------------------|------|-----------------------|----------------------------|-------|-----|---------------------------|-----|------|
| 00                       | NULL | (Null character)      | 32                         | space | 64  | @                         | 96  | `    |
| 01                       | SOH  | (Start of Header)     | 33                         | !     | 65  | A                         | 97  | a    |
| 02                       | STX  | (Start of Text)       | 34                         | "     | 66  | B                         | 98  | b    |
| 03                       | ETX  | (End of Text)         | 35                         | #     | 67  | C                         | 99  | c    |
| 04                       | EOT  | (End of Trans.)       | 36                         | \$    | 68  | D                         | 100 | d    |
| 05                       | ENQ  | (Enquiry)             | 37                         | %     | 69  | E                         | 101 | e    |
| 06                       | ACK  | (Acknowledgement)     | 38                         | &     | 70  | F                         | 102 | f    |
| 07                       | BEL  | (Bell)                | 39                         | '     | 71  | G                         | 103 | g    |
| 08                       | BS   | (Backspace)           | 40                         | (     | 72  | H                         | 104 | h    |
| 09                       | HT   | (Horizontal Tab)      | 41                         | )     | 73  | I                         | 105 | i    |
| 10                       | LF   | (Line feed)           | 42                         | *     | 74  | J                         | 106 | j    |
| 11                       | VT   | (Vertical Tab)        | 43                         | +     | 75  | K                         | 107 | k    |
| 12                       | FF   | (Form feed)           | 44                         | ,     | 76  | L                         | 108 | l    |
| 13                       | CR   | (Carriage return)     | 45                         | -     | 77  | M                         | 109 | m    |
| 14                       | SO   | (Shift Out)           | 46                         | .     | 78  | N                         | 110 | n    |
| 15                       | SI   | (Shift In)            | 47                         | /     | 79  | O                         | 111 | o    |
| 16                       | DLE  | (Data link escape)    | 48                         | 0     | 80  | P                         | 112 | p    |
| 17                       | DC1  | (Device control 1)    | 49                         | 1     | 81  | Q                         | 113 | q    |
| 18                       | DC2  | (Device control 2)    | 50                         | 2     | 82  | R                         | 114 | r    |
| 19                       | DC3  | (Device control 3)    | 51                         | 3     | 83  | S                         | 115 | s    |
| 20                       | DC4  | (Device control 4)    | 52                         | 4     | 84  | T                         | 116 | t    |
| 21                       | NAK  | (Negative acknowl.)   | 53                         | 5     | 85  | U                         | 117 | u    |
| 22                       | SYN  | (Synchronous idle)    | 54                         | 6     | 86  | V                         | 118 | v    |
| 23                       | ETB  | (End of trans. block) | 55                         | 7     | 87  | W                         | 119 | w    |
| 24                       | CAN  | (Cancel)              | 56                         | 8     | 88  | X                         | 120 | x    |
| 25                       | EM   | (End of medium)       | 57                         | 9     | 89  | Y                         | 121 | y    |
| 26                       | SUB  | (Substitute)          | 58                         | :     | 90  | Z                         | 122 | z    |
| 27                       | ESC  | (Escape)              | 59                         | ;     | 91  | [                         | 123 | {    |
| 28                       | FS   | (File separator)      | 60                         | <     | 92  | \                         | 124 |      |
| 29                       | GS   | (Group separator)     | 61                         | =     | 93  | ]                         | 125 | }    |
| 30                       | RS   | (Record separator)    | 62                         | >     | 94  | ^                         | 126 | ~    |
| 31                       | US   | (Unit separator)      | 63                         | ?     | 95  | _                         |     |      |
| 127                      | DEL  | (Delete)              |                            |       |     |                           |     |      |
| 128                      | Ç    |                       | 160                        | á     | 192 | Ł                         | 224 | Ó    |
| 129                      | ü    |                       | 161                        | í     | 193 | ł                         | 225 | ô    |
| 130                      | é    |                       | 162                        | ó     | 194 | Ł                         | 226 | õ    |
| 131                      | â    |                       | 163                        | ú     | 195 | ł                         | 227 | ö    |
| 132                      | ä    |                       | 164                        | ñ     | 196 | Ł                         | 228 | ø    |
| 133                      | à    |                       | 165                        | Ñ     | 197 | ł                         | 229 | ő    |
| 134                      | ã    |                       | 166                        | ª     | 198 | Ł                         | 230 | µ    |
| 135                      | ç    |                       | 167                        | º     | 199 | ł                         | 231 | þ    |
| 136                      | ê    |                       | 168                        | ¿     | 200 | Ł                         | 232 | ß    |
| 137                      | ë    |                       | 169                        | ®     | 201 | ł                         | 233 | Ù    |
| 138                      | è    |                       | 170                        | ¬     | 202 | Ł                         | 234 | Ú    |
| 139                      | ï    |                       | 171                        | ½     | 203 | ł                         | 235 | Û    |
| 140                      | î    |                       | 172                        | ¼     | 204 | Ł                         | 236 | Ü    |
| 141                      | ï    |                       | 173                        | ⅓     | 205 | ł                         | 237 | Ý    |
| 142                      | Ä    |                       | 174                        | «     | 206 | Ł                         | 238 | Ÿ    |
| 143                      | Å    |                       | 175                        | »     | 207 | ł                         | 239 | ·    |
| 144                      | É    |                       | 176                        | ⋈     | 208 | Ł                         | 240 | ≡    |
| 145                      | æ    |                       | 177                        | ⋈     | 209 | ł                         | 241 | ±    |
| 146                      | Æ    |                       | 178                        | ⋈     | 210 | Ł                         | 242 | ≡    |
| 147                      | ô    |                       | 179                        | ⋈     | 211 | ł                         | 243 | ¾    |
| 148                      | ö    |                       | 180                        | ⋈     | 212 | Ł                         | 244 | ¶    |
| 149                      | ò    |                       | 181                        | À     | 213 | ł                         | 245 | §    |
| 150                      | û    |                       | 182                        | Â     | 214 | Ł                         | 246 | ÷    |
| 151                      | ù    |                       | 183                        | Ã     | 215 | ł                         | 247 | °    |
| 152                      | ÿ    |                       | 184                        | ©     | 216 | Ł                         | 248 | °    |
| 153                      | Ö    |                       | 185                        | ⋈     | 217 | ł                         | 249 | °    |
| 154                      | Ü    |                       | 186                        | ⋈     | 218 | Ł                         | 250 | °    |
| 155                      | ø    |                       | 187                        | ⋈     | 219 | ł                         | 251 | °    |
| 156                      | £    |                       | 188                        | ⋈     | 220 | Ł                         | 252 | °    |
| 157                      | Ø    |                       | 189                        | φ     | 221 | ł                         | 253 | °    |
| 158                      | ×    |                       | 190                        | ¥     | 222 | Ł                         | 254 | °    |
| 159                      | f    |                       | 191                        | ſ     | 223 | ł                         | 255 | nbsp |

# Các kiểu dữ liệu cơ bản (tt)

Kiểu chuỗi (xâu ký tự)

- Một dãy các ký tự đứng liền nhau được gọi là 1 xâu ký tự.
- Ngôn ngữ C hiểu ký tự '\0' là ký tự kết thúc xâu  
=>Chỉ lưu biết địa chỉ của ký tự đầu tiên của chuỗi
- Một hằng xâu ký tự đặt trong cặp dấu “nháy kép”

Ví dụ:

“Ho Chi Minh” là hằng xâu ký tự

|   |   |  |   |   |   |  |   |   |   |   |    |
|---|---|--|---|---|---|--|---|---|---|---|----|
| H | o |  | C | h | i |  | M | i | n | h | \0 |
|---|---|--|---|---|---|--|---|---|---|---|----|

Khai báo biến xâu: **char** **tênbiếnxâu**[**số byte**];

# Kiểu mảng - *Array*

---

## Định nghĩa:

- **Mảng** là một kiểu dữ liệu có cấu trúc do người lập trình định nghĩa
- Dùng biểu diễn các đối tượng dữ liệu ở dạng một dãy các thành phần có cùng kiểu với nhau (**kiểu cơ sở**)
- **NNLT C** luôn chỉ định một khối nhớ liên tục cho một biến kiểu mảng
- **Kích thước của mảng** được xác định ngay khi khai báo và không bao giờ thay đổi

# Khai báo *array* trong C

## Khai báo:

**kiểu cơ sở** TênBiến [**Số phần tử**];

## Ví dụ:

- `int a[100];`  
*//a là mảng biểu diễn dãy gồm 100 số nguyên int*
- `float b[10];`
- `long int big[100];`
- `double d[100];`
- `long double v[SIZE];`
- `int a[5] = { 10, 20, 30, 40, 50};`
- `double d[] = { 1.5, 2.7};`

# Kiểu *array* – Lưu ý

- ❑ Các thành phần của mảng được truy xuất thông qua **chỉ số của chúng 0..n-1**
- ❑ Thao tác truy xuất **không kiểm tra giới hạn của chỉ số**

```
int main()
{
    int    a[6];
    int    i = 7;
    a[0] = 59;
    a[5] = -10;
    a[i/2] = 2;
    a[6] = 0;
    a[-1] = 5;
    return 0;
}
```

a

|     |   |
|-----|---|
| 59  | 0 |
|     | 1 |
|     | 2 |
| 2   | 3 |
|     | 4 |
| -10 | 5 |





# Ép kiểu (casting)

## Cú pháp

- **(kiểu\_mới)** biểu thức hoặc **kiểu\_mới (biểu thức)**

Ví dụ:

```
#include <stdio.h>
#include <conio.h>
int main(void)
{
    int i;
    float x;

    i = (int)10.46;    /* i = 10 */
    x = (float)1/3;    /* x = 1.0/3 = 0.33333 */

    getch();

    return 0;
}
```

# Đặc tả định dạng

---

- %d: đọc vào một số nguyên int
- %u: đọc vào một số nguyên unsigned int
- %x: đọc vào một số nguyên hệ 16
- %o: đọc vào một số nguyên hệ bát phân
- %s: đọc vào một chuỗi ký tự
- %c: đọc vào một ký tự đơn char
- %f: đọc vào một số thực float
- %e: số chấm động (ký hiệu có số mũ)
- %hd: đọc vào một số nguyên short int
- %lf, %lu: đọc vào một số thực double

# Xuất/Nhập dữ liệu

---

## □ Xuất dữ liệu

- In ra màn hình
- Xuất ra máy in
- Xuất ra các cổng

## □ Nhập dữ liệu

- Nhập từ bàn phím
- Nhập từ file
- Nhập từ các cổng

# Xuất ra màn hình – Hàm *printf*

❑ Thuộc thư viện "**stdio.h**"

❑ Cú pháp:

**printf**("Xâu điều khiển" [, danh sách tham số]);

❑ Ví dụ:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    printf("hello, world\n");

    getch();

    return 0;
}
```

# Hàm printf

---

□ Xâu điều khiển bao gồm 3 loại kí tự

– Các kí tự điều khiển

- \n sang dòng mới
- \f sang trang mới
- \b xóa kí tự bên trái
- \t dấu Tab

– Các kí tự để đưa ra màn hình

– Các kí tự định dạng và khuôn in

- Ví dụ:            % . 2 f            % 8 d            % s            % - 5 c

# Hàm printf

---

## In các ký tự đặc biệt

### ❑ Công thức định dạng

**% [-] [w] [.p] Kt**

- **W**: wide (độ rộng)
- **P**: precse (độ chính xác)
- **Kt**: Ký tự chuyển dạng

### ❑ Giá trị của w

- Nhỏ hơn hoặc bằng độ rộng thực tế => in bình thường
- Lớn hơn độ rộng thực tế (chèn thêm khoảng trống cho đủ độ rộng là w (nếu có dấu trừ chèn bên phải, ngược lại chèn bên trái)

### ❑ Giá trị p: quy định làm tròn đến bao nhiêu chữ số đằng sau chấm thập phân.

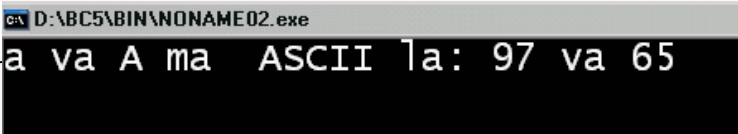
# Hàm printf

- Ví dụ

```
#include <stdio.h>
#include <conio.h>
void main(void){

    printf("%c va %c ma  ASCII la: %d va %d", 'a', 'A', 'a', 'A');

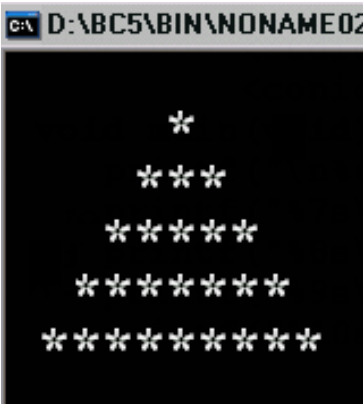
    getch();
}
```



c:\ D:\ABC5\BIN\NONAME02.exe  
a va A ma ASCII la: 97 va 65

```
#include <stdio.h>
#include <conio.h>
void main(void){
    printf("\n%6s\n", "*");
    printf("%7s\n", "****");
    printf("%8s\n", "*****");
    printf("%9s\n", "*****");
    printf("%10s\n", "*****");

    getch();
}
```



c:\ D:\ABC5\BIN\NONAME02.exe  
 \*  
 \*\*\*  
 \*\*\*\*\*  
 \*\*\*\*\*  
 \*\*\*\*\*  
 \*\*\*\*\*

# Hàm puts

---

- ❑ Thư viện: “**stdio.h**”
- ❑ Công dụng: in ra màn hình 1 xâu ký tự
- ❑ Ví dụ:

```
#include <stdio.h>
#include <conio.h>

void main(void) {
    puts("\n      *");
    puts("\n     ***");
    puts("\n    *****");
    puts("\n   ********");
    puts("\n  *********");
    puts("\n *********");
    puts("\n*****");

    getch();
}
```



# Hàm putchar()

---

- ❑ Thư viện: “**stdio.h**”
- ❑ Ý nghĩa: Dùng để in một kí tự lên màn hình.
- ❑ Cú pháp: `putchar(ch);`
  - Ch: chứa mã của kí tự cần in ra màn hình
- ❑ Ví dụ:

```
putchar( 'A' );
```

# Hàm sprintf

---

- ❑ Thư viện: “**stdio.h**”
- ❑ Công dụng: giống printf, chỉ khác ở điểm kết quả không xuất ra màn hình mà đưa vào 1 biến xâu.
- ❑ Ví dụ:

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    char buffer[80];

    sprintf(buffer, "Gia tri cua PI = %f\n", M_PI);
    puts(buffer);

    return 0;
}
```

# Nhập dữ liệu, hàm: *scanf*

---

- Thuộc thư viện "**stdio.h**"
- Cú pháp:

**scanf**("**Xâu điều khiển**", <**Địa chỉ biến**>) ;

- Xâu điều khiển: Cho phép định dạng dữ liệu nhập vào
- Danh sách tham số: Là địa chỉ các biến cần nhập liệu sử dụng toán tử **&** để xác định địa chỉ các biến.
- Ví dụ:

```
scanf ("%d%f%c", &x, &y, &z) ;
```

```
scanf ("%s", &hoten) ;
```

# Hàm scanf

---

- ❑ Không thông báo cho người dùng biết giá trị cần nhập là giá trị gì? => Phải kết hợp với lệnh xuất dữ liệu
- ❑ Không nhận được xâu có dấu cách trống
- ❑ Không xóa ký tự thừa trong bộ đệm => ảnh hưởng tới lệnh nhập phía sau.

```
#include <stdio.h>
#include <conio.h>

int main(void){
    char ht[40];

    scanf("%s", &ht);
    printf("Ho ten vua nhap: %s", ht);

    getch();
    return 1;    |
}
```

# Sử dụng *fflush* sau *scanf*

## Công dụng:

- xóa những ký tự thừa còn dư lại trong bộ đệm của ***scanf***

```
#include <stdio.h>
#include <conio.h>

int x, y;
float u, z;

void main(void)
{
    printf("Nhap 2 so x, y:");
    scanf("%2d%d", &x, &y);

    fflush(stdin);    //xoa ky tu thua
    printf("Nhap tiep 2 so z, u:");
    scanf("%f%f", &z, &u);

    printf("x=%d, y=%d\n", x, y);
    printf("z=%f, u=%g\n", z, u);

    getch();
}
```

# Kết hợp nhập và xuất dữ liệu

```
#include <stdio.h>
#include <conio.h>
const luongcb = 730000;
float luong, bacluong;
char hoten[40];

void main()
{
    printf("Nhap ho ten nhan vien:");
    scanf("%s", &hoten);

    fflush(stdin);
    printf("Nhap bac luong:");
    scanf("%f", &bacluong);

    luong = luongcb*bacluong;
    printf("Nhan vien %s linh %.2f (d/thang)",
           hoten, luong);

    getch();
}
```

# Hàm *gets*

- ❑ Thư viện: “conio.h”
- ❑ Công dụng: nhập vào 1 xâu ký tự
- ❑ Ví dụ:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    char string[80];

    printf("Input a string:");
    gets(string);
    printf("The string input was: %s\n", string);

    return 0;
}
```



Liệu có dùng  
gets nhập 1  
số không?

# Ví dụ

```
#include <stdio.h>
#include <conio.h>
int N, soblock;
float t, x;

void main()
{
    printf("Nhap so giay:");
    scanf("%d", &N);

    soblock = (N%6!=0)?1:0;
    soblock = soblock + N/6;
    printf("So block la: %d", soblock);

    printf("\nGia tien 1 block:");
    scanf("%f", &x);

    t = soblock*x;
    printf("So tien phai tra la: %f", t);
    getch();
}
```

```
#include <stdio.h>
#include <conio.h>
const luongcb = 730000;
float luong, bacluong, BHYT, BHTN, DP;
char hoten[40];

void main(){
    printf("Nhap ho ten nhan vien:");
    gets(hoten);
    printf("Nhap bac luong:");
    scanf("%f", &bacluong);
    luong = luongcb*bacluong;
    BHYT = 0.015*luong;
    BHTN = 0.01*luong;
    DP = 0.01*luong;
    luong = luong - BHYT - BHTN - DP;

    printf("NV %s linh %.2f(d/thang)",
           hoten, luong);
    getch();
}
```



# Một số hàm nhập/xuất dữ liệu khác

| Hàm                       | Ý nghĩa                                                  |
|---------------------------|----------------------------------------------------------|
| <code>int getch()</code>  | đọc nhập 1 ký tự từ bàn phím (không hiển thị ở màn hình) |
| <code>int getche()</code> | giống getch() nhưng hiển thị ở màn hình                  |
| <code>int putch()</code>  | In ra màn hình 1 ký tự                                   |
| <code>int kbhit()</code>  | Kiểm tra xem có 1 phím bị nhấn không?                    |
| <code>cprintf()</code>    | Giống printf nhưng có màu sắc                            |
| <code>gotoxy(x, y)</code> | chuyển tới vị trí (cột x, hàng y)                        |

# Bài tập - Nhập/xuất dữ liệu

1. Viết chương trình cho nhập các hệ số của phương trình bậc nhất 1 ẩn  $ax+b=0$  (với  $a \neq 0$ ) hãy in ra màn hình nghiệm của phương trình đã cho.
2. Viết chương trình hiển thị ra màn hình số tiền phải trả cho 1 cuộc gọi điện thoại N giây. Biết rằng cước gọi tính theo mỗi block là xđ, và **1 block bằng 6 giây**. Các giá trị x, N nhập từ bàn phím.
3. Viết chương trình nhập tọa độ của 3 điểm A, B, C trên mặt phẳng. Tính và in ra màn hình chu vi, diện tích của tam giác ABC.
4. Viết chương trình nhập họ tên, hệ số lương của 1 nhân viên. Tính và in ra màn hình số tiền còn lĩnh của nhân viên đó sau khi trừ đi các khoản: BHYT 1.5%, BH thất nghiệp 1% và Đảng phí 1%.
5. Viết chương trình hiện ra màn hình thể tích của tứ diện ABCD biết rằng A, B, C có tọa độ nhập từ bàn phím  $(X_A, Y_A)$   $(X_B, Y_B)$   $(X_C, Y_C)$  và chiều cao từ đỉnh D xuống mặt phẳng ABC là  $h$ .

# Các cấu trúc điều khiển

---

❑ Khái niệm khối lệnh

❑ Cấu trúc rẽ nhánh

– Rẽ nhánh **if**

– Rẽ nhánh **switch....case**

❑ Cấu trúc lặp

– Lặp theo biến đếm – Lặp dạng **for**

– Lặp theo điều kiện

• Dạng **while**

• Dạng **do....while**

– Câu lệnh **break** và **continue**

# Khái niệm khối lệnh

- ❑ Khối lệnh gồm nhiều lệnh đặt trong cặp dấu { }

- Ví dụ:

```
{  
    printf("Nhap so gia tri a[%d]", i);  
    scanf("%d", &a[i]);  
}
```

- Khối lệnh thường được dùng trong các cấu trúc điều khiển. Nếu muốn 1 cấu trúc điều khiển tác động lên nhiều lệnh thì các lệnh đó phải đặt trong 1 khối lệnh
- Các khối lệnh có thể viết lồng nhau (lưu ý sử dụng cách viết theo cấu trúc khối cho dễ phân biệt)

# Cấu trúc rẽ nhánh

---

❑ Rẽ nhánh dạng **if**

❑ Cú pháp:

- **if** ( điều kiện)  
    <lệnh thực hiện>;
- **if** ( điều kiện)  
    <lệnh 1>;  
    **else**  
    <lệnh 2>;

❑ Công dụng:

- Thực hiện 1 trong 2 lệnh tùy thuộc vào điều kiện đúng/sai.
- Nếu thực hiện nhiều hơn 1 lệnh => phải để trong cặp dấu **{ }**

# Rẽ nhánh *if* - ví dụ

---

- Kiểm tra số chẵn, lẻ:

```
1 #include<stdio.h>
2 #include<conio.h>
3 int main() {
4     int a;
5     printf("Nhap a: ");
6     scanf("%d", &a);
7     if(a%2==0)
8         printf("a la so chan");
9     else
10        printf("a la so le");
11     return 0;
12 }
```

# Toán tử luận lý

- Sử dụng các toán tử luận lý để xây dựng các điều kiện phức tạp hơn

| Toán tử | Ký hiệu           |
|---------|-------------------|
| AND     | <b>&amp;&amp;</b> |
| OR      | <b>  </b>         |
| NOT     | <b>!</b>          |

| A            | !A           |
|--------------|--------------|
| <b>TRUE</b>  | <b>FALSE</b> |
| <b>FALSE</b> | <b>TRUE</b>  |

| A     | B     | A && B      |
|-------|-------|-------------|
| TRUE  | TRUE  | <b>TRUE</b> |
| TRUE  | FALSE | FALSE       |
| FALSE | TRUE  | FALSE       |
| FALSE | FALSE | FALSE       |

| A     | B     | A    B       |
|-------|-------|--------------|
| TRUE  | TRUE  | TRUE         |
| TRUE  | FALSE | TRUE         |
| FALSE | TRUE  | TRUE         |
| FALSE | FALSE | <b>FALSE</b> |

# Bài tập *if*

---

- Kiểm tra 1 số nhập vào có phải là số chẵn lớn hơn 10 hoặc số lẻ nhỏ hơn 10.
- Kiểm tra các cạnh nhập vào có tạo thành một tam giác hay không?
- Biện luận giải phương trình bậc 2 một ẩn.



# Ví dụ

---

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
void main(){
    float xa, ya, xb, yb, xc, yc;
    float a, b, c;

    printf("Nhap toa do diem A(xa, ya):");
    scanf("%f%f", &xa, &ya);
    printf("Nhap toa do diem B(xb, yb):");
    scanf("%f%f", &xb, &yb);
    printf("Nhap toa do diem C(xc, yc):");
    scanf("%f%f", &xc, &yc);

    a = sqrt( (xb-xc)*(xb-xc) + (yb-yc)*(yb-yc) );
    b = sqrt( (xa-xc)*(xa-xc) + (ya-yc)*(ya-yc) );
    c = sqrt( (xa-xb)*(xa-xb) + (ya-yb)*(ya-yb) );

    if(a==b && b==c)
        printf("Tam giac A, B, C deu\n");
    else
        printf("Tam giac A, B, C Khong deu\n");

    getch();
}
```

# Sử dụng khối lệnh trong if

## □ Lưu ý:

- Nếu sau **if** hoặc **else** tác động lên nhiều hơn 1 lệnh thì phải sử dụng cặp dấu { }

```
if(delta == 0) {  
    x0 = (float)-b/(2*a);  
    printf("Phuong trinh co nghiem kep x0=%g", x0);  
}  
else {  
    x1 = (-b - sqrt(delta))/(2*a);  
    x1 = (-b + sqrt(delta))/(2*a);  
    printf("Hai nghiem: x1=%.2f va x2=%.2f", x2, x2);  
}
```

- Nếu sau **if** có dấu ; thì điều kiện không còn tác động đến lệnh phía sau

```
int a = 5;  
if( a < 3 );  
    printf("%d nho hon 3", a);
```

Kết  
quả?

# Điều kiện trong rẽ nhánh *if*

## □ Lưu ý:

- Giá trị số cũng được coi là điều kiện (số = 0: điều kiện sai; số ≠ 0: điều kiện đúng)

```
if(delta = 0) {  
    x0 = (float)-b/(2*a);  
    printf("Phuong trinh co nghiem kep x0=%g", x0);  
}  
else {  
    x1 = (-b - sqrt(delta))/(2*a);  
    x1 = (-b + sqrt(delta))/(2*a);  
    printf("Hai nghiem: x1=%.2f va x2=%.2f", x2, x2);  
}
```

Delta=0?

# Sử dụng *else if*

- Khi muốn chọn 1 trong n quyết định thì sử dụng *else if* như sau:

```
if (điều kiện 1)
    khối lệnh 1;
else if (điều kiện 2)
    khối lệnh 2;
.....
.....
else if (biểu thức n-1)
    khối lệnh n-1;
else
    khối lệnh n;
```

## *Viết chương trình xếp loại học lực theo điểm trung bình học tập.*

```
#include <stdio.h>
#include <conio.h>
void main(void) {
    float diem;
    printf("Nhap diem vao");
    scanf("%f", &diem);

    if (diem < 5)
        printf("Xep loai: kem");
    else if (diem < 7)
        printf("Xep loai: trung binh");
    else if (diem < 8)
        printf("Xep loai: kha");
    else if (diem < 9)
        printf("Xep loai: gioi");
    else
        printf("Xep loai: xuat sac");

    getch();
}
```

# Sử dụng toán tử ? thay if

- Cú pháp:

(điều kiện) ? lệnh 1 : lệnh 2 ;

- Công dụng:

- Lệnh 1 được thực hiện nếu điều kiện đúng
- Lệnh 2 được thực hiện nếu điều kiện sai

- Ví dụ:

```
1 #include<stdio.h>
2 #include<conio.h>
3 int main() {
4     int a=7,b=10,max;
5     (a>b) ? max=a:max=b;
6     printf("max la: %d",max);
7     return 0;
8 }
```

# Rẽ nhánh *switch ... case...*

- ❑ Rẽ nhánh *if* chỉ cho phép chọn một trong hai phương án
- ❑ Rẽ nhánh *switch ... case* cho phép chọn một trong nhiều phương án.
- ❑ Cú pháp:

```
switch (biểu thức) {  
    case  $n_1$ :  
        Các câu lệnh;  
    case  $n_2$ :  
        Các câu lệnh;  
    ...  
    case  $n_k$ :  
        Các câu lệnh;  
    [default: Các câu lệnh;]  
}
```

# Rẽ nhánh switch ... case...

## □ Công dụng

- Nếu biểu thức nguyên có giá trị bằng **nhãn  $n_i$**  thì máy sẽ nhảy đến thực hiện các lệnh của nhãn đó, nếu không thì máy sẽ nhảy đến thực hiện các lệnh trong thành phần tùy chọn **default**
- Máy sẽ ra khỏi toán tử **switch** khi nó gặp câu lệnh **break**, **return** hoặc nó gặp dấu **“}”** của câu lệnh **switch**
- Chú ý, khi máy nhảy tới **nhãn  $n_i$** , nếu kết thúc dãy lệnh trong nhãn này không có câu lệnh **break** hoặc **return** thì máy sẽ tiếp tục thực hiện các lệnh trong **nhãn  $n_{i+1}$**
- Thường cuối mỗi dãy lệnh của một nhãn có một lệnh **break**



# Rẽ nhánh switch ... case...

```
0 #include<conio.h>
1 #include<stdio.h>
2 int main(){
3     int a=10;
4     switch(a%2){
5         case 0:
6             printf("a la so chan");
7             break;
8         case 1:
9             printf("a la so le");
10            break;
11    }
12    return 0;
13 }
```

```
0 #include<conio.h>
1 #include<stdio.h>
2 int main(){
3     int a=10;
4     switch(a%2){
5         case 0:
6             printf("a la so chan");
7             break;
8         default:
9             printf("a la so le");
10            break;
11    }
12    return 0;
13 }
```

# Cấu trúc lặp

---

❑ Một công việc nào đó được thực hiện lặp đi lặp lại nhiều lần

❑ Ví dụ

– In ra màn hình các số từ 1 đến 10, mỗi số trên một dòng

• Cách 1:

```
printf("1\n");
```

```
printf("2\n");
```

...

```
printf("10\n");
```

• Cách 2:

– Lặp i chạy từ 1 tới 10

```
printf("%d\n", i);
```



**In ra 1000 dòng?  
Viết lại 1000 dòng!**

# Lặp dạng for

– Cú pháp

```
for([B.Thức 1]; [B.thức 2]; [B.Thức 3])  
    <Lệnh cần lặp>;
```

- Các phần trong cặp dấu “[” và “]” là không bắt buộc
- Các dấu “;” và cặp ngoặc “(” và “)” bắt buộc phải có

– Ví dụ:

```
1 #include<stdio.h>  
2 int main() {  
3     for(int i=0; i<10;i++)  
4         printf("Hello class \n");  
5 }
```

# Lặp dạng *for*

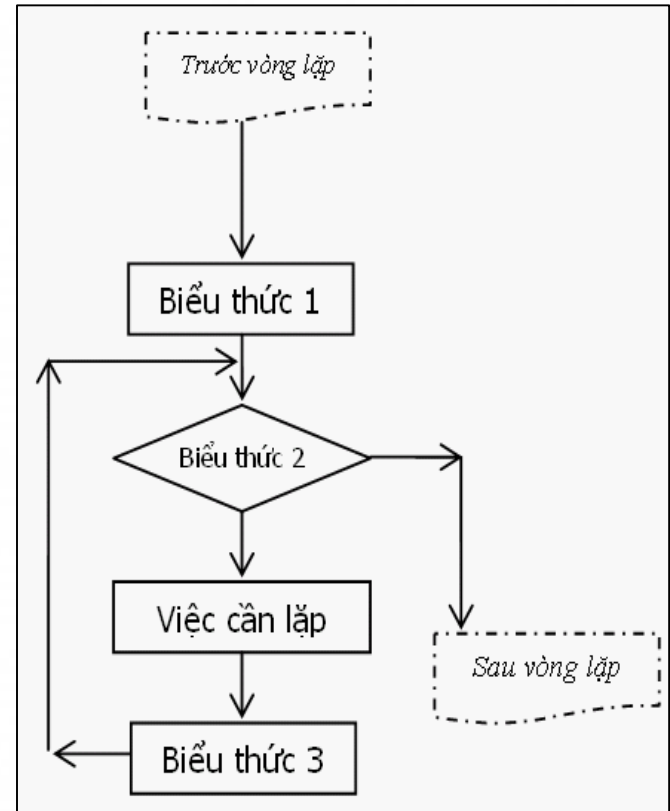
Các bước hoạt động:

1. Tính **B.Thức 1**.

2. Tính **B.Thức 2**.

- Nếu sai => thoát vòng lặp.
- Nếu đúng => đi vào thực hiện việc cần lặp

3. Tính **B.Thức 3**, sau đó quay trở lại bước 2 để bắt đầu các bước lặp mới.



# Lặp dạng *for*

- ❑ **Biểu thức 1:** Thường là 1 phép gán để khởi tạo giá trị ban đầu cho biến điều khiển
- ❑ **Biểu thức 2:** Thường là 1 biểu thức điều kiện
- ❑ **Biểu thức 3:** Cũng là 1 phép gán để thay đổi giá trị biến điều khiển (biểu thức tăng hoặc giảm)

```
#include <stdio.h>
#include <conio.h>
int i, S=0;
void main(void) {
    //Truoc vong lap
    for (i = 1; i<=15; i++){
        if (i%2)
            S += i;
    }
    //sau vong lap
    printf("Tong cac so chan 1..15 = %d", S);

    getch();
}
```

# Lặp dạng *for*

---

## ❑ Nhận xét

- Biểu thức 1 chỉ được tính một lần
- Biểu thức 2, biểu thức 3 và khối lệnh trong thân lệnh **for** được lặp đi lặp lại nhiều lần
- Dựa các giá trị khởi tạo biến điều khiển, điều kiện lặp và biểu thức 3 có thể tính được số lần lặp.

## ❑ Khi biểu thức 2 vắng mặt thì nó được xem là đúng (vòng lặp vô hạn).

- Để thoát khỏi lệnh **for** trong trường hợp này phải dùng lệnh **break** hoặc **return**.

## ❑ Có thể sử dụng các lệnh **for** lồng nhau.

# Lặp *for* trên dữ liệu kiểu *array*

```
int a[100];
for(i=1; i<=N; i++)
{
    printf("Nhap a[%d]=", i);
    scanf("%d", &a[i]);
}
```

```
int a[100];
for(i=0; i<N; i++)
{
    printf("Phan thu thu %d\n", i);
    printf("%5d", a[i]);
}
```

# Lặp dạng *while*

---

## □ Lệnh lặp **while**

### – Cú pháp

**while** (**biểu thức**)  
    khối lệnh cần lặp;

### – Ý nghĩa:

- Trong khi **biểu thức** có giá trị đúng (khác 0) thì còn phải thực hiện khối lệnh. Việc lặp dừng lại khi biểu thức có giá trị sai (bằng 0).
- Lặp **while** kiểm tra điều kiện trước khi thực hiện khối lệnh.



# Lặp dạng *while* – Ví dụ

```
/*  
    Tính tổng 1+2+3+...+N  
*/  
  
#include <stdio.h>  
void main(){  
    int n, s, i;  
    printf("N = "); scanf("%d", &n);  
    s = 0;  
    i = 1;  
    while (i <= n) {  
        s = s + i;  
        i++;  
    }  
    printf("Tong cac so 1..N = %d\n", n, s);  
    getch();  
}
```

# Lặp dạng *while* – Lưu ý

## ❑ Nhận xét

- Biểu thức điều kiện luôn được đặt trong cặp dấu "(" và ")"
- Biểu thức điều kiện sẽ được tính toán đầu tiên nên phải có giá trị xác định

## ❑ Câu lệnh sau làm gì ?

```
while (0) printf("nothing\n");
```

# Lặp dạng *do...while*

## ❑ Cú pháp

```
do{  
    các lệnh của  
    thân vòng lặp;  
} while (biểu thức);
```

## ❑ Ý nghĩa

- Thực hiện khối lệnh trong khi biểu thức có giá trị đúng (khác 0)
- Thực hiện khối lệnh trước khi kiểm tra biểu thức điều kiện
- Khối lệnh được thực hiện ít nhất 1 lần

# Lặp *do...while* - ví dụ

---

```
0 //Nhap vao 1 so lon hon 0
1 #include<conio.h>
2 #include<stdio.h>
3 int main() {
4     int n;
5     do{
6         printf("Nhap n: ");
7         scanf("%d", &n);
8     }while (n<=0);
9     printf("Gia tri cua n la: %d",n);
10    return 0;
11 }
```

# Ví dụ

---

```
0 //Nhap vao 1 so lon hon 0
1 #include<conio.h>
2 #include<stdio.h>
3 int main() {
4     int n;
5     while (n<=0) {
6         printf("Nhap n: ");
7         scanf("%d", &n);
8     }
9     printf("Gia tri cua n la: %d", n);
10    return 0;
11 }
```



Kết quả?

# Ví dụ

```
0 #include<stdio.h>
1 int main() {
2     int i=0;
3     do{
4         printf("Hello %d",i);
5         i--;
6     }while(i>0);
7     return 0;
8 }
```



**Kết quả?**

```
0 #include<stdio.h>
1 int main() {
2     int i=0;
3     while(i>0) {
4         printf("Hello %d",i);
5         i--;
6     }
7     return 0;
8 }
```

# ***break và continue***

---

- ❑ Lệnh **break** thường được sử dụng kết hợp lệnh lặp
  - Dùng để thoát khỏi vòng lặp một cách bất thường (không quan tâm tới điều kiện lặp)
  - Nếu có nhiều lệnh lặp lồng nhau thì lệnh **break** chỉ thoát vòng lặp trực tiếp chứa nó
- ❑ Lệnh **continue**
  - Dùng để quay trở lại từ đầu vòng lặp
  - Thực hiện lần lặp mới mà không đi hết các lệnh còn lại trong thân vòng lặp

# Bài tập

---

1. Nhập vào một dãy  $a_1, a_2, \dots, a_N$ .

Tính **TBC** các số lẻ trong dãy số  $a_1, a_2, \dots, a_N$ .

Tìm phần tử chẵn đầu tiên của dãy số trên (Làm 2 cách: dùng **while**, dùng **for**))

2. Cho dãy điểm  $M_1(x_1, y_1), M_2(x_2, y_2), \dots, M_n(x_n, y_n)$  trên mặt phẳng. Hãy:

- Tìm độ dài đường gấp khúc  $M_1M_2 \dots M_n$
- Tìm đoạn  $M_iM_j$  ( $i \neq j$ ) có độ dài lớn nhất
- Đếm xem có bao nhiêu đoạn cắt trục  $Oy$
- Có bao nhiêu điểm thuộc góc phần tư thứ nhất.



# Bài tập (tiếp)

---

3. Viết chương trình tính tổng của chuỗi:

$$H(n) = 1 + 1/2 + 1/3 + \dots + 1/n$$

Bằng cách sử dụng cấu trúc **while**

4. Hãy viết chương trình tìm trong số các phần tử của dãy  $a_1, a_2, \dots, a_N$  có bao nhiêu cặp  $(a_i, a_j)$  với  $i \neq j$  thỏa điều kiện  $a_i + a_j = x$